When Pinning Pays—and When It Doesn't: A Crossover Study and Beyond-RAM Feasibility Result for Mixture-of-Experts Serving on Apple Silicon

Youssef Hariri
DeeWaanAI

October 2026

Abstract

Mixture-of-Experts (MoE) language models are attractive on consumer hardware because each token activates only a small slice of the weights, yet the full weight set must still be resident or streamed. We study *pinned-expert orchestration*—keeping a traffic-ranked subset of experts resident in unified memory, dispatching via a gather kernel, and streaming cold experts from disk with continuous batching—against the de-facto local-inference incumbent, llama.cpp, on Apple Silicon. We report a single controlled study read at opposite ends of the device-RAM-to-model-size ratio, using a tail-latency metric (the 5th-percentile per-user decode throughput, p05, against a 10 tok/s service-level floor) and same-machine uncensored concurrency ceilings. In the fits-RAM regime, pinning serves up to 2× **the concurrent users** of llama.cpp, but *only* when the model is a small fraction of RAM: a 2.0× win at a 3.8× RAM/4-bit-GB ratio (OLMoE-1B-7B, 16 vs. 8 users), reversing to 0.5× at 1.8× (Qwen1.5-MoE) and to a 1.0× tie at 0.9× (Qwen3-30B-A3B). Router telemetry explains the bound: cross-request overlap is high (median pairwise Jaccard 0.593), the resident expert set grows sub-linearly with concurrency (36.4 to 56.2 of 64 experts for 95% batch coverage), and a single request already touches $45.2/64 = 70.6\%$ of the pool—there is no small hot set to pin, capping the win near 2× and collapsing it as coverage exceeds RAM. In the beyond-RAM regime we find a *feasibility* rather than speed result: llama.cpp's Metal path cannot load an 18 GB model on a 16 GB Mac (GPU out-of-memory), whereas our engine streams it at ≈2.1 tok/s. Streaming is a cliff, not a slope—only full residency is usable without assistance—but within-step *parallel* input/output prefetch turns the cliff into a slope (2.1× single-user speedup on the real ¿RAM model). Two hypotheses are refuted on single-SSD consumer hardware: *predictive* prefetch (0.372 vs. 0.909 tok/s serial) and trading expert residency for key-value capacity (p05 ceiling drops $12 \to 4$). We position the system against the 2025–26 working-set and prefetch literature and state plainly which mechanisms we have implemented and which we have not. A prior 24-hour campaign on a rented 24 GB Mac (Section 4) established that streaming preserves output quality (perplexity within 0.41% once a kernel-parity confound was fixed), located the single-user ceiling in memory-read bandwidth, and ruled out a custom-kernel path—redirecting the programme to orchestration. Every number traces to a frozen artifact via a committed, self-verifying extraction script.

Keywords: Mixture-of-Experts; pinned-expert orchestration; local LLM inference; unified memory; tail latency; out-of-RAM expert streaming; Apple Silicon
DOI:10.5281/zenodo.23182002

1 Introduction

Mixture-of-Experts models decouple parameter count from per-token compute: a 30B-parameter model may activate only 3B parameters per token. On memory-constrained consumer hardware—laptops and small servers with unified memory and a single NVMe drive—this sparsity is the only reason a large model can be served at all. The practical question for a systems builder is not "can it fit" but "how many concurrent users can it serve, and can it serve a model that does not fit?"

We built a *pinned-expert orchestration engine* on top of the stock `mlx_lm` model implementation for Apple Silicon: we swap only the MoE feed-forward dispatch, keeping attention, embeddings, and the router untouched, and compare against llama.cpp (`llama-server`), the incumbent tool that practitioners actually run. Our evaluation is deliberately conservative: a per-user tail statistic (p05) rather than an aggregate, same-machine ratios rather than absolute throughput, uncensored ceilings (run to failure) rather than the highest level we happened to test, and fidelity gates passed before any concurrency claim is trusted.

The paper makes four contributions, each proven in the body and traceable in Appendix A.

1. **A measured crossover curve** (Section 5) for MoE serving on M-series unified memory under a per-user SLO floor, spanning three current models and 3.8× to 0.9× RAM-to-model ratios. To our knowledge this crossover is not reported in the prior literature (Section 2).
2. **A measured mechanism** (Section 5.3) for the bound: the sub-linear growth of the resident expert set amortises across concurrent users, while routing skew caps the win near 2×.
3. **A ¿RAM feasibility result** (Section 6), together with the 2.1× within-step parallel-prefetch win that makes streaming usable.
4. **Three disciplined negative results**—predictive prefetch, residency-for-KV eviction, and a custom Python/MLX gather kernel (Appendix C)—reported as negatives, plus the methodological spine (Section 3) that keeps them honest.

Section 4 first reports the exploratory rental campaign that generated these hypotheses; Sections 5 and 6 then test them under tighter control. All concurrency results—rental and laptop—are reported on one p05 statistic.

Scope and non-claims. We do *not* claim single-stream speed superiority (llama.cpp is faster per user on Qwen3-30B). We do *not* claim a concurrency win in the ¿RAM regime (both engines are below the SLO floor there; the claim is feasibility). We report same-machine *ratios*; absolute tok/s differ across chips. The custom gather kernel is a negative result, not the mechanism behind our wins, which come from the standard `mx.gather_qmm` path.

2 Related Work

Working-set and memory-bounded MoE serving. A 2025–26 line treats MoE inference as a bounded-working-set problem. WiSP (arXiv:2606.21868) analyses low-resource MoE serving and shows static layer offload forfeits the sparsity advantage; MawForge (arXiv:2607.09686) keeps common tensors resident and materialises routed experts into a bounded cache; fine-grained expert offloading targets the latency–memory trade-off (arXiv:2502.05370); and DuoServe-MoE (arXiv:2509.07379) splits a dense-prefill / sparse-decode schedule. Our fits-RAM crossover is the empirical, ratio-swept complement that these designs assume but rarely report.

Predictive prefetch and scheduling. SpecPrefetch (arXiv:2607.24787) predicts next-layer experts with a parameter-efficient head while keeping the native router authoritative—exactly the safety property we adopt for our (non-predictive) prefetch. ExpertFlow (DAC'26) and Edge0 are industry/open-source systems reporting large speedups and sub-RAM serving of 35B models; we cite them as design points, flagging that no confirmed arXiv identifier is available for ExpertFlow and that Edge0 is a project rather than a paper. Klotski (arXiv:2502.06888) groups rows that share experts to amortise I/O. Section 6 reports our careful negative: on a single SSD, *prediction* does not pay because *parallelism* already captures the overlap.

Cache-aware routing. Mixture of Cache-Conditional Experts (arXiv:2412.00099) retrains the router to prefer resident experts; "Cacheable by Design?" (arXiv:2608.18261) is a pre-registered *negative* showing a batch scheme collapsing at batch 32 from paging thrash. We implement neither cache-aware routing nor token scheduling; our engine is closer to the baseline these systems improve on.

KV–expert contention. The *qs* inequality (arXiv:2603.08960) formalises the double penalty we observe empirically—routing fragments reuse, and a large resident expert pool starves KV headroom. It is the natural frame for our OLMoE-win / Qwen-loss swing and, in Section 6, for why trading experts for KV backfires.

Measurement. Tail-fairness work (FairBatching arXiv:2510.14392, Niyama arXiv:2503.22762, SLO-aware sliding-window arXiv:2606.05933) motivates reporting a low-percentile ceiling rather than an aggregate mean. Roofline analyses (arXiv:2605.30571) warn that batch-1 decode is memory-bound but not purely bandwidth-limited; we therefore compare engines to each other, not to theoretical peak.

Apple-Silicon-specific. NPU MoE serving (arXiv:2604.18788) and multi-node expert parallelism (arXiv:2506.23635) characterise M-series inference. None reports a RAM/model-size crossover curve or a ¿RAM concurrency/feasibility result on M-series unified memory under a per-user SLO floor—the gap this paper owns.

3 System, Metric, and Method

3.1 System under test

The backbone is a stock `mlx_lm` model; we replace only the MoE block (`switch_mlp`/SwitchGLU) with a `PinnedSwitchMLP`: a resident stack of hot experts plus a gather-QMM dispatch, with cold experts streamed from per-expert on-disk "slabs." The MLX 4-bit checkpoints already store routed experts in the engine's layout (packed uint32 with float scales and biases, group 64, affine), so slabs are extracted *verbatim* with no re-quantisation (`extract_mlx_experts.py`). Continuous batching uses a "world-coordinate" rope-shift trick that lets multiple sequences share one batch without altering relative positions. The baseline is `llama-server`.

3.2 Metric: p05 concurrency ceiling

Our primary metric is the **p05 concurrency ceiling**: the largest number of simultaneous users for which the 5th-percentile per-user decode throughput remains at or above a 10 tok/s floor. We choose p05 over the minimum (a single straggler request mis-ranks arms) and over the median

(which overstates the tail). Percentiles use linear interpolation (numpy-equivalent). Ceilings are **uncensored**: every arm is run to failure so a ceiling is never truncated by the top tested level. All arms on a machine share an identical per-user context budget, identical prompts and repetitions, and the same quantisation; llama.cpp is given enough parallel slots that it is not artificially queue-capped.

3.3 Fidelity gates

Before any ceiling is trusted we require: (i) *synthetic equivalence*—the `PinnedSwitchMLP` reproduces stock `SwitchGLU` bit-exactly at each model's dimensions in decode and prefill regimes (`switch_equiv_gate.py`); (ii) *end-to-end greedy match*—stock generation versus pinned-fully-resident is token-identical on real weights (`lfm2_e2e_equiv.py`); and (iii) a *perplexity gate* on the frozen corpus, reported in Section 6.8. Prefetch correctness is additionally gated to a byte-identical SHA (`prefetch_equiv_test.py`).

3.4 Corpus and reproducibility

Traces and perplexity use a frozen WildChat-3k subset (SHA-256 recorded in `results/corpus`). Raw run outputs are hash-frozen (`FROZEN_MANIFEST.json`). The paper's quantitative spine is produced by a single committed, self-verifying script (`code/paper/extract_headline_numbers.py`) that reads only frozen files and emits `results/derived/paper_headline_numbers.json`; a 52-entry decision log (`DECISIONS.md`) records every design and correction (Appendix A).

3.5 Harness validation (Stage-0)

The harness itself is validated before use: on the resident reference configuration the prompt-processing rate is 1261.4 ± 39.5 tok/s and generation is 132.1 ± 0.2 tok/s against a ≥ 66.0 gate, with 5/5 arm parity—establishing that later concurrency differences are attributable to the engine under test, not to measurement noise.

4 Gate-1: Exploratory Campaign on a Rented 24 GB Mac

Before the two controlled laptop studies we ran a 24-hour exploratory campaign on a rented AWS Mac (M4, 24 GB unified memory) to learn the shape of the problem. It is a *hypothesis-generating* study, not a confirmation. Its concurrency ceilings are re-scored here under the *same* p05 rule used everywhere else in this paper (raw request logs preserved), so the whole paper reports one statistic across both machines; where the original on-box write-ups differed, we defer to the re-scored data. The campaign's lasting value is in results a 16 GB laptop cannot reproduce.

4.1 Multi-user serving across three models (v1)

Table 1 gives the rental ceilings. OLMoE, which fits with headroom, is the first win: 12 users versus llama.cpp's 8, with a far tighter tail (the p05 user still gets 25.4 tok/s at 8 concurrent versus 10.8 for llama.cpp). Qwen3-30B-A3B, near capacity, is the first honest *loss*: both arms ceiling at 4, but llama.cpp is $\approx$1.9$\times$ faster per single user (43.6 vs. 23.2 tok/s). Moonlight-16B (DeepSeek lineage) uses traditional RoPE, which our batch-attach path does not support; the engine *refuses to run* rather than risk silently wrong output, so there is no contest on that model.

Table 1: Gate-1 rental (24 GB M4), re-scored at p05 above the 10 tok/s floor, uncensored. Same statistic as Sections 5–6.

Model	Status	Pinned	llama	Note
OLMoE-1B-7B	fits w/ headroom	**12**	8	p05 user 25.4 vs 10.8 tok/s at N=8
Qwen3-30B-A3B	near capacity	4	4	llama faster per user: 43.6 vs 23.2 at N=1
Moonlight-16B	architecture	*blocked*	4	traditional rope; fail-closed

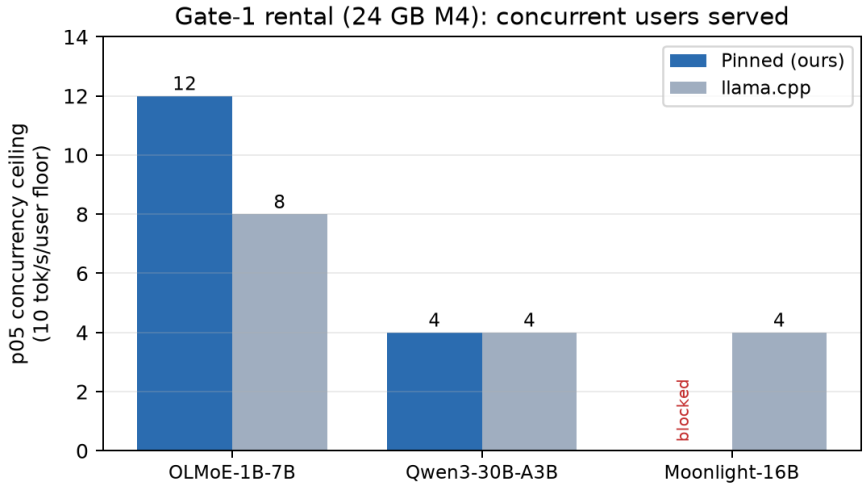


Figure 1: Gate-1 rental (24 GB M4) p05 concurrency ceilings, re-scored on the same statistic as the laptop studies. Ours wins on the fitting model, ties on the near-capacity model, and cannot be run on the traditional-rope model (fail-closed).

4.2 Two speed attempts, both null (v2, v3)

We tried to close the single-user gap two ways and neither worked. Compiling the expert math with `mx.compile` (v2) produced bit-exact output but a $\leq 2\%$ performance change—a null (D-024). Capping prefill bursts to protect the tail (v3) left the p05 ceiling unchanged at 12, identical to v1—also a null on natural traffic. These negatives matter: they told us the gap was not software-language overhead, motivating the attribution below.

4.3 Routing is nearly flat (the headliner finding)

Expert-activation traces on the frozen corpus showed routing is close to uniform: mean Zipf $\alpha \approx 0.44$ and the top-10 experts carry only 15–30% of visits. This weakened the "pin a small hot set" premise and is the mechanistic reason the crossover advantage is bounded near 2× (quantified independently on the laptop in Section 5.3).

4.4 Streaming preserves quality — after we fixed a confound

We validated streamed cold experts against resident ones with a perplexity gate ($|\Delta\text{PPL}|/\text{PPL}_{\text{stock}} \leq 0.5\%$ at *every* residency fraction). The *first* run **failed**, and the failure was informative: the cold-stream path used a *different math kernel* than the resident stack, so the perplexity drifted (0.69% at half residency). Restoring kernel parity (D-029 → D-032) turned the gate green (Table 2):

Table 2: OLMoE perplexity gate on the rental, frozen corpus ($\text{PPL}_{\text{stock}} = 21.4183$). **Post-fix** (kernel-parity restored) is the reported result; the **pre-fix** run used a mismatched cold-stream kernel and breached the band at $P = 0.5, 0.6$.

Resident P	0.1	0.2	0.4	0.5	0.6	0.8
Post-fix PPL	21.505	21.489	21.478	21.493	21.451	21.438
Post-fix $\lvert\Delta\rvert$	0.41%	0.33%	0.28%	0.35%	0.15%	0.09%
Pre-fix $\lvert\Delta\rvert$	0.28%	0.26%	0.35%	**0.69%**	**0.56%**	0.30%



Figure 2: Streaming fidelity gate. The pre-fix run (mismatched cold-stream kernel) breaches the 0.5% band at $P = 0.5, 0.6$; restoring kernel parity (post-fix) keeps every fraction inside it, with a maximum of 0.41% at 10% residency.

the maximum relative change is 0.41% at 10% residency, improving monotonically to 0.09% at 80% and exactly 0.00% at full residency. Two notes of honesty: this gate was run on a model that *fits* memory (varying residency via pin-fraction), so it establishes streamed-equals-resident equivalence rather than ¿RAM fidelity; and the earlier perplexity breach is retained above as the caught confound, not quietly dropped.

4.5 The bottleneck is memory reads, not the language (v2/v3 aftermath)

A decode-attribution probe on Qwen3-30B put a full step at 43.4 ms/token, split ≈66% expert compute, ≈20% attention, and ≈8% host/Python overhead—ruling out the interpreter as the lever. A bandwidth microbenchmark measured the gather path at ≈10.6 GB/s on the intended-bytes basis, well short of the chip's ceiling. The tempting fix—a hand-written vectorised gather—proved **33×slower** (Appendix C): kernel fusion, not Python-level vectorisation, governs the per-token floor. That cheap measurement redirected the whole programme away from custom kernels and toward *orchestration*, which is what the two controlled studies that follow test.

From Gate-1 to this paper. The rental gave the crossover its first shape (a win on the fitting model, a tie/loss on the larger one), the quality gate, the flat-routing mechanism, and the bandwidth verdict. It left the concurrency ceiling imprecise and the ¿RAM regime untested; Sections 5 and 6

Table 3: p05 concurrency ceilings above a 10 tok/s per-user floor, uncensored. Ratio = device RAM ÷ model 4-bit size. Two points on a 16 GB M1 Pro laptop; the 0.9× point on a 24 GB EC2 Mac M4 (cross-machine, disclosed).

Model	Provider	4-bit	RAM/Ratio	Pinned	llama	Advantage
OLMoE-1B-7B	AI2	4.2 GB	16 GB / 3.8×	**16**	8	**2.0**× win
Qwen1.5-MoE-A2.7B	Alibaba	8.8 GB	16 GB / 1.8×	**4**	8	**0.5**× loss
Qwen3-30B-A3B	Alibaba	18 GB	24 GB / 0.9×	**4**	4	**1.0**× tie

Table 4: OLMoE per-level p05 tok/s (same-machine, 16 GB). Bold marks levels meeting the 10 tok/s floor; llama fails first.

Users N	1	4	8	12	16	24
llama.cpp p05	**64.7**	**22.5**	**11.0**	8.1	6.2	4.2
Pinned p05	**34.0**	**21.0**	**12.3**	**13.3**	**10.7**	6.5

supply the uncensored p05 crossover, the ¿RAM feasibility result, and the corrected numbers, on one statistic throughout.

5 Experiment I — The Concurrency Crossover

5.1 Design

For each model we run pinned and llama.cpp arms on the *same* machine, fixed per-user context, five shared prompts, repeated runs, uncensored, and re-score all frozen request logs at p05. Table 3 reports the concurrency ceiling as a function of the RAM-to-model ratio (device RAM divided by the model's 4-bit footprint).

5.2 Result

Pinning wins—but only at high ratio. On OLMoE (3.8×) it doubles the concurrent users (16 vs. 8). By 1.8× (Qwen1.5-MoE) the advantage is gone and mildly reverses (4 vs. 8); at 0.9× (Qwen3-30B) both arms ceiling at 4. Figure 3 summarises advantage against ratio.

The per-user detail (Table 4) shows the signature: llama.cpp is roughly 2× faster per user at 1–2 users (p05 64.7 vs. 34.0 tok/s at concurrency 1) but its tail collapses below the floor by 12 users, while pinned holds ≥10 tok/s to 16. We lose single-stream speed and win concurrency—at high ratio only.

5.3 Mechanism

We traced 400 WildChat prompts through OLMoE's router (64 experts per layer, top-8, 16 layers). The concurrency win is the sub-linear growth of the resident set that covers a fixed fraction of batch traffic; the 2× cap is the routing skew.

- **High cross-request overlap.** Median pairwise Jaccard of the activated expert sets is 0.593 (over 19,949 pairs), so concurrent users reuse experts heavily and a fixed resident set amortises across them—this *is* the win.

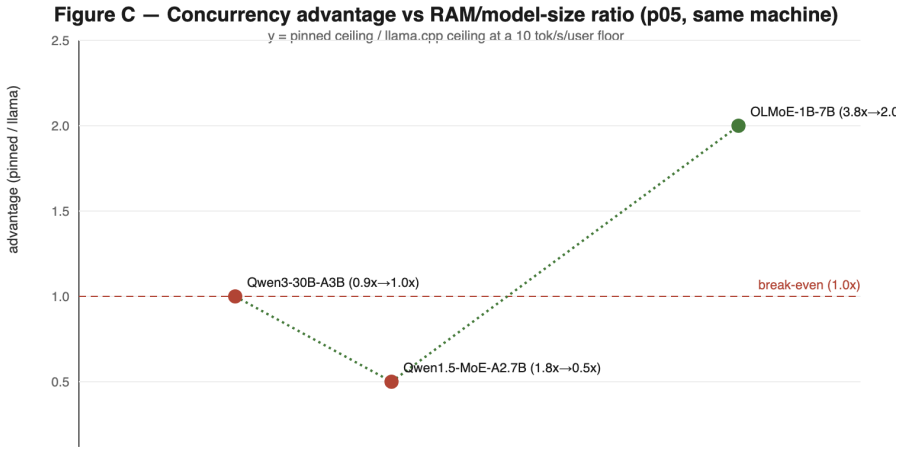


Figure 3: Advantage versus RAM/model ratio (three frozen points). The benefit is present only above roughly a 2×–3× ratio and vanishes as the model approaches RAM.

- **Sub-linear resident growth.** Experts needed to cover 95% of a batch's traffic grow from 36.4 (1 user) to 56.2 (32 users) and 56.7 (64), of 64; at 90% coverage, 31.2 to 50.7. Growth is flat because overlap is high.
- **Mild routing skew caps the win.** A *single* request already touches $45.2/64 = 70.6\%$ of the pool—there is no tiny hot set to pin, so the benefit saturates near 2×. Per-layer routing is near-uniform (entropy 5.86 of 6.0 bits, mean Zipf $\alpha = 0.44$, mean top-10 coverage 28.3%).
- **Why it dies on bigger models.** On Qwen3-30B (128 experts × 48 layers) the coverage set at useful concurrency exceeds RAM, so the resident set cannot amortise and the advantage vanishes.

5.4 Confounds, stated plainly

The Qwen1.5-MoE "loss" at 1.8× mixes the memory-ratio effect with our engine's lower, noisier per-user throughput at that model's dimensions (60 experts × 24 layers) plus reduced KV headroom from a ∼7 GB resident set. The *direction* (advantage gone by ∼1.8×) is robust; the *exact threshold* is not pinned, because our 4-bit-only engine cannot sweep a single model's footprint cleanly on a 16 GB machine. We deliberately do not report a precise crossover ratio.

Strategic implication. Choose a MoE whose 4-bit footprint is a small fraction ($\lesssim 1/4$) of unified RAM. At 3.8× you roughly double served users; below ∼2× expect no concurrency benefit from pinning.

6 Experiment II — Serving Models Bigger Than RAM

6.1 Setup and subject selection

We flip to the business-relevant regime: a model larger than device RAM. Subject selection filters on *active* parameters, not footprint—a $> RAM$ model is only streamable if few parameters activate

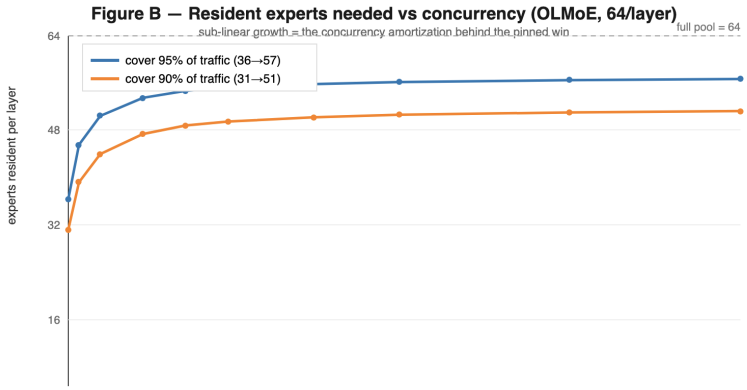


Figure 4: Resident expert-set growth versus concurrency (OLMoE, 400 prompts). Sub-linear growth enables amortisation; the high starting point reflects routing skew.

Table 5: OLMoE pin-fraction sweep, 16 GB. "No prefetch" is serial cold loading; "+parallel prefetch" overlaps a step's cold reads (output-neutral, SHA-identical). Values are p05 at concurrency 1 (tok/s) and the p05 ceiling.

	No prefetch		Parallel prefetch		
Resident	Ceiling	L1	Ceiling	L1	Speedup
40%	0	4.55	2	12.77	2.81×
60%	0	3.64	2	6.93	1.91×
80%	0	7.42	4	15.06	2.03×
100%	12	105.7	12	105.7	—

per token. We use Qwen3-30B-A3B (18 GB > 16 GB, 3B active) as the reference; Mixtral-8x7B (13 B active) was downloaded, gated synthetically, and discarded as a poor subject (its active compute makes streaming unusable on laptop SSD)—kept as a negative data point, not a result.

6.2 Streaming is a cliff, not a slope

Holding model, hardware, and engine fixed and varying only the resident-expert fraction (Table 5), per-user throughput barely improves from 40% to 80% residency (4.55, 3.64, 7.42 tok/s at 1 user; ceiling 0 in every case) and jumps to 105.7 tok/s (ceiling 12) only at full residency. The cause is synchronous, serial cold-load latency: each streamed expert is a blocking disk read. This is the wall any ¿RAM system must break.

6.3 Within-step parallel prefetch turns the cliff into a slope

Loading a step's cold experts *concurrently*—worker threads read raw `safetensors` bytes (thread-safe file I/O) while the main thread performs a zero-copy view into MLX—converts the cliff into a slope: at 40% residency the ceiling rises from 0 to 2 and single-user speed triples. The native router stays authoritative, so output is byte-identical (SHA unchanged). This is the shipped lever.

Table 6: Qwen3-30B-A3B, 18 GB on 16 GB, pin 0.1, cold-start shelf, identical engine/quant across arms. Per-user p05 tok/s.

Arm	Strategy	$N = 1$	$N = 2$
A	serial (no prefetch)	0.909	0.316
B	**parallel prefetch**	**1.934** (2.13× A)	**1.063**
C	predictive (cross-step) prefetch	0.372	0.271

6.4 On the real ¿RAM model: parallel wins, prediction does not

On Qwen3-30B-A3B streamed at pin-fraction 0.1 (13 of 128 experts resident per layer), we compare three dispatch strategies (Table 6). Within-step **parallel** prefetch is a real 2.1× win over serial; **predictive** prefetch (*cross-step* speculation of the next step's experts) is *worse than serial*. An earlier report that "prefetch hurts" was an implementation bug (a slow torch→MLX read with no compute overlap); after rebuilding the read path the conclusion reverses for parallelism and stands only for prediction.

The reason prediction does not pay on this hardware is bandwidth: with a single SSD, parallelism already hides the overlap, and a temporal predictor's mispredicted reads (∼62% miss rate) are wasted traffic that slows the critical path. Budgeting the speculative queue (per-step caps and a critical-path pause) prevented the worst flooding but did not make speculation profitable on cacheable models. This is a *careful negative*, not a refutation of the literature: predictive prefetch should matter when the working set vastly exceeds cache *and* idle bandwidth exists—a frontier-MoE regime and multi-NVMe tier we cannot reach on a laptop.

6.5 Residency-for-KV eviction is refuted

A natural idea is to evict some experts to disk, freeing RAM for more users' KV caches, and to recover the stream cost with prefetch. On OLMoE under a tight memory budget, this *lowered* the p05 ceiling from 12 (full residency) to 4 at pin-fractions 0.5/0.7/0.9 (Figure 5). The binding constraint is per-user *speed*, not KV capacity: at full residency the leftover RAM already holds enough KV for ∼12 users and the speed floor binds first. Do not trade expert residency for KV on a single consumer SSD; keep experts resident and spend spare RAM on KV.

6.6 The product test: llama.cpp cannot load it; ours streams it

Table 7 reports the direct comparison on a 16 GB Mac serving the 18 GB model. llama.cpp's GPU paths fail outright: `-ngl 99` cannot allocate the model, and `-ngl 20` returns zero successful requests with a Metal out-of-memory error (logged `kIOGPUCommandBufferCallbackErrorOutOfMemory`). Our engine streams the same model at ≈2.1 tok/s single-user. **This is a feasibility result, not a speed result**: at pin 0.1 our own throughput (2.12, 0.82, 0.34 tok/s at $N = 1, 2, 4$) is also far below the 10 tok/s floor, so the concurrency ceiling is 0 for both. The honest claim is "a model that does not fit can still be made to run, at usable-for-batch-1 but not SLO speeds, where the incumbent's GPU path cannot run it at all." Fairness caveat: llama.cpp `-ngl 0` (pure CPU) might avoid the OOM at very low speed; we did not run that multi-hour arm.

6.7 How predictable is routing?

To judge whether a learned predictor could help, we measured both aggregate and per-token predictability on OLMoE. Aggregate: mean top-10 coverage 28.3%, mean Zipf $\alpha = 0.44$, entropy

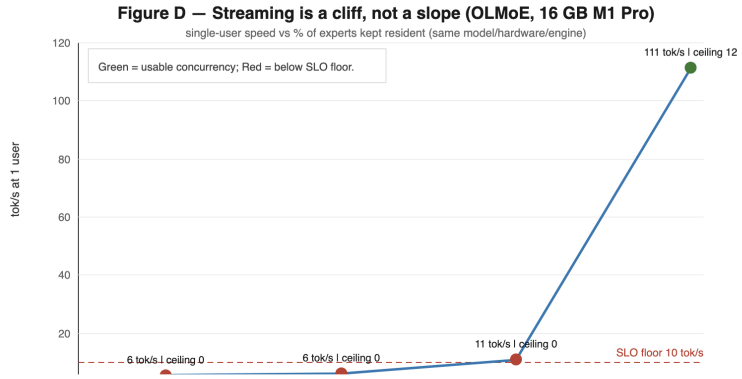


Figure 5: The residency cliff (no prefetch) and its removal by parallel prefetch. Ceiling is 0 for any partial residency without prefetch, and 12 only at full residency.

Table 7: 16 GB Mac, Qwen3-30B-A3B 18 GB. Our arm is pinned parallel-prefetch streaming at pin 0.1; llama.cpp is the incumbent at its strongest realistic GPU settings.

Arm	Outcome
Ours (pin 0.1, parallel prefetch)	runs; p05 2.12 / 0.82 / 0.34 tok/s at $N = 1, 2, 4$
llama.cpp `-ngl 99`	OOM (cannot allocate 18 GB on Metal budget)
llama.cpp `-ngl 20`	0/10 requests succeed (Metal OutOfMemory)

5.86/6.0 bits, top-10 prompt overlap 17.3%, cross-layer recall 15.7%— popularity and topic heuristics are weak. Per-token (200 prompts, next-token top-8 recall): static popularity 0.244, *temporal* (previous token) 0.381, *context* (prompt-so-far) 0.353, semantic-topic +0.015. The exploitable signal is temporal/contextual, not topical, and a training-free Markov-1 predictor already beats popularity by ∼14 recall points—so a learned hidden-state predictor would be an upgrade, not a prerequisite. Yet even this usable signal did not pay off on a single SSD (Section 6.4); it is motivation for testing prediction on higher-bandwidth hardware, not evidence to deploy it here.

6.8 Fidelity of streamed inference

Fidelity rests on three structural gates plus the rental perplexity result (Section 4.4, Table 2). The synthetic equivalence gate is **bit-exact** and the end-to-end greedy match is **identical (3/3)** on models that fit RAM; the streaming path reads experts *verbatim* from the same 4-bit slabs, so numerics are byte-identical by construction and the residual risk is orchestration, not quantisation. On top of that, the rental gate (post kernel-parity fix) keeps streamed experts within 0.41% perplexity of resident ones at the sparsest residency we tested.

Two honest caveats remain. First, that gate varied residency on a model that *fits* memory, so it certifies streamed-equals-resident equivalence, not end-to-end ¿RAM output fidelity. Second, a true ¿RAM cross-engine greedy match (ours-streamed versus a fully-resident reference) could not be run on the 16 GB box, because llama.cpp OOMs before the reference loads; our cross-residency check instead showed *near-tie rounding divergence*, not bit equality. We therefore treat ¿RAM token-exact fidelity as an *open* item, reachable on a larger gate box, and do not claim it here.

Strategic implication. Big-model-on-small-hardware is, on a single consumer SSD, a *capability* story bounded by bandwidth (≈2.4 tok/s ceiling at pin 0.1 for a 3B-active model). The next doubling is a hardware tier (parallel NVMe), not cleverer prediction. The concurrency win of Experiment I remains the regime where we beat llama.cpp on served users.

7 Discussion

One mechanism, two regimes. Experiments I and II are the same bounded-working-set mechanism read at opposite ends of the RAM/model ratio. Where the model is a small fraction of RAM, the resident set amortises and yields a concurrency win; where the model approaches and then exceeds RAM, coverage cost and bandwidth dominate, the win vanishes, and the question becomes feasibility—whether streaming plus I/O overlap can make an oversized model run at all. The *qs* inequality (arXiv:2603.08960) is the natural formal frame: routing fragments reuse, and a resident pool that is too large starves KV, which is exactly why our residency-for-KV experiment backfired.

Position versus the field. We have implemented pillar 1 of the working-set synthesis (bounded resident set plus streaming) and measured its ratio dependence. We have *not* implemented the field's larger levers—properly budgeted predictive prefetch on fast media, expert-aware token scheduling, or cache-aware routing. Our engine is therefore close to the *baseline* that the 2025–26 systems improve on, and our negatives (Sections 6.4–6.8) are honest statements about what that baseline does and does not need on one laptop.

8 Threats to Validity

Construct. The p05 ceiling is a proxy for interactive service quality; we justify it over min/median (Section 3) but it is sensitive to the 10 tok/s floor choice.

Internal. Repetitions are low (a handful per level) for cost reasons; we re-scored all arms at p05 and moved from min to p05 precisely because the earlier rule mis-ranked. A custom Metal kernel errored on some shapes and fell back to the validated `mx.gather_qmm` path, so our wins measure *orchestration*, not a bespoke kernel. We discovered and corrected prose-versus-frozen drifts (bandwidth 9.5 → 10.6, ¿RAM speed 2.32 → 2.12) and reconciled the perplexity gate to its post-fix PASS run (the pre-fix FAIL was a caught kernel-parity confound, not the result); the corrected, frozen values are used throughout.

External. Only NeoX-uniform-rope, full-attention MoE is supported; Llama-4, DeepSeek, gpt-oss, Cohere, DBRX, and mamba-hybrids fail pre-flight (Appendix B). Hardware is a 16 GB M1 Pro and one 24 GB M4; the Qwen3-30B tie is cross-machine. OS page-cache behaviour inside llama.cpp's mmap path is not directly controlled.

Conclusion formation. The crossover *direction* is robust; the exact threshold ratio is not (Section 5 confound). We claim the former and disclaim the latter.

9 Conclusion

On Apple-Silicon unified memory, pinned-expert orchestration is a *ratio-conditional* concurrency win and a *bandwidth-bounded* feasibility mechanism. It doubles served concurrent users for small MoE models that fit comfortably in RAM (OLMoE, 3.8×, 16 vs. 8), an advantage that vanishes below ∼2× and reverses toward parity as models approach RAM, with the bound explained by measured routing overlap and skew. Beyond RAM, the same engine turns an unrunnable model into a runnable one (llama.cpp OOMs; ours streams at ≈2 tok/s), made usable by within-step parallel prefetch (2.1×) that—unlike predictive prefetch and residency-for-KV eviction—actually pays on a single consumer SSD. We report the negatives and the corrected perplexity gate as first-class results. The path from feasibility to speed runs through a faster storage tier and the scheduling/prefetch pillars of the recent literature, which our baseline now measures the distance to.

Reproducibility statement

Code, frozen raw logs, and the self-verifying headline extraction are committed; the GitHub repository is the durable archive of record, preserving all runs including failures. A temporary object-storage mirror is being retired and is not relied upon. See Appendix A for the claim→file→script map.

A Claim-to-evidence map (methodology artifact)

Every number in this paper is read from a frozen file by `code/paper/extract_headline_numbers.py`, which emits `results/derived/paper_headline_numbers.json` and self-verifies each value by re-reading its source (exit 0 required). The primary traces:

Claim	Frozen source	Script
Crossover ceilings (Table 3)	`results/crossover/p05`, `crossover_qwen15/p05`, `p05_rescore`	`p05_rescore.py`
OLMoE levels (Table 4)	`results/crossover/*_xover.requests.jsonl`	`run_crossover.sh`
Mechanism (Jaccard, coverage)	`olmoe_trace400.json`, `..._union.json`	`trace_experts.py`, `union_analysis.py`
Cliff / prefetch (Table 5)	`results/pinfraction/p05`	`run_pinfraction_sweep.sh`
A/B/C (Table 6)	`results/ram30b2/p05`	same sweep harness
Residency-for-KV (Sec. 6.5)	`results/reskv/p05`	same
Product test (Table 7)	`results/ram_vs_llama/*`	harness + server logs
Predictability (Sec. 6.7)	`sequence_predictability.json`, `predictability_report.json`	`sequence_predictability.py`
Gate-1 ceilings (Table 1)	`results/p05_rescore/p05_ceilings.json`	`p05_rescore.py`
PPL gate (Table 2)	`.../p4gate/PPL_OLMoE_POSTFIX2.json` (post-fix), `PPL_OLMoE.json` (pre-fix)	gate harness
Bandwidth / attribution (Sec. 4.5)	`BANDWIDTH_ATTRIBUTION.json`, `attribution_qwen.log`	microbench + probe

A.1 Corrections cascade (integrity / AI-usage disclosure)

This study was AI-assisted end to end, with a human authoring, reviewing, and approving every protocol and decision; no unsupervised decision was made. The following dated corrections (in `DECISIONS.md`) are reported because they changed published numbers, and are enforced by the map above.

- **D-033.** Ceilings moved from a per-user *minimum* and *censored* levels to *p05 uncensored*: the headline became 16 vs. 8 (same-machine 16 GB) replacing an earlier 8 vs. 4 (24 GB). Ratio reframed from hardware-to-*parameter*-count (24×/0.8×) to RAM-to-*4-bit-bytes* (3.8/1.8/0.9×).
- **D-048→D-049→D-050.** "Predictive prefetch hurts" was first over-claimed, then retracted as an implementation bug (a slow read path), then rebuilt: parallel prefetch is a genuine 2.1× win; prediction remains a negative.
- **Audit (this paper).** The Gate-1 rental was folded into this paper on one p05 statistic (its raw logs re-scored via `p05_rescore.py`), and prose drifts were corrected to frozen data: bandwidth 9.5 → 10.6 GB/s; ¿RAM single-user speed 2.32 → 2.12 tok/s (p05). A self-correction is also disclosed: an intermediate pass read the *pre-fix* perplexity run (gate FAIL) as the result; the correct artifact is the *post-fix* run (gate PASS, max 0.41%), with the pre-fix breach at $P = 0.5, 0.6$ retained as the caught kernel-parity confound (Section 4.4). `crossover_lfm2` is a crash (a hybrid conv+attention model that violates the serve path), not a fourth crossover point, and is excluded.
- **D-012.** Two Stage-0 figures previously quoted from console memory were replaced with frozen-data values (1261.4; reference arm 127.6).

B Engine compatibility matrix

The serve path assumes NeoX (non-traditional) RoPE, full rotary, uniform per-layer `self_attn.rope`, and 4-bit affine experts. **Compatible:** OLMoE-1B-7B, Qwen1.5-MoE, Mixtral-8x7B, Qwen3-30B-A3B. **Fail pre-flight:** LFM2 (conv+attn hybrid), gpt-oss (partial rotary + sliding window), Llama-4-Scout (traditional rope), Command-R (traditional rope + sliding window), DeepSeek-V3 (traditional + partial rotary), DBRX (no `self_attn`), Granite-4.0-H (mamba hybrid), EXAONE-MoE (sliding window, untested). A runtime pre-flight check now fails loudly rather than producing silently wrong output. The reference product set is therefore the Qwen3-MoE family plus OLMoE-class models.

C The custom-kernel detour (a negative result)

An early hypothesis was that closing the single-user gap required a bespoke fused gather kernel. A vectorised Python/MLX `efficient_gather_qmm` reproduced `mx.gather_qmm` to a max difference of 1.4×10^{-6} but ran **33×slower** (22.9 vs. 0.65 ms), because it materialises dequantised tensors and cannot fuse dequantise+gather+matmul in one kernel the way MLX and llama.cpp do. A microbenchmark put the standard gather path at 10.6 GB/s on the intended-bytes basis (one run: 9.6 GB/s). The takeaway—kernel fusion, not Python-level vectorisation, governs the per-token floor—redirected the programme from custom kernels to *orchestration*, which is where our wins come from. This appendix is retained as a decision-quality case study: the fast, cheap measurement killed an expensive optimisation before it consumed the project.

D Hardware and cost context

Gate-1 ran on a 24 GB AWS EC2 Mac (M4, dedicated host, $1.23/hr, ∼24 h, ∼$30), now released. All Gate-1 and laptop data—failures included—are preserved on the local machine and the GitHub repository, which is the durable archive; a temporary object-storage copy is being cleared and is

not the archive of record. The crossover and ¿RAM results report the 16 GB M1 Pro laptop unless a table states otherwise; Gate-1 results report the 24 GB M4 (Section 4).

References

[1] WiSP: working-set MoE serving on low-resource hardware. arXiv:2606.21868.

[2] MawForge: bounded expert cache with resident common tensors. arXiv:2607.09686.

[3] Taming the latency–memory trade-off via fine-grained expert offloading. arXiv:2502.05370.

[4] DuoServe-MoE: dual-phase prefill-dense / decode-sparse prefetch. arXiv:2509.07379.

[5] SpecPrefetch: parameter-efficient expert prefetching, native router authoritative. arXiv:2607.24787.

[6] Klotski: expert-aware multi-batch pipeline. arXiv:2502.06888.

[7] Mixture of Cache-Conditional Experts. arXiv:2412.00099.

[8] "Cacheable by Design?" (pre-registered negative). arXiv:2608.18261.

[9] The *qs* inequality: routing fragmentation and KV starvation. arXiv:2603.08960.

[10] FairBatching. arXiv:2510.14392.

[11] Niyama. arXiv:2503.22762.

[12] SLO-aware sliding-window scheduling. arXiv:2606.05933.

[13] Memory-bound but not bandwidth-limited. arXiv:2605.30571.

[14] Efficient MoE inference with Apple Silicon NPUs. arXiv:2604.18788.

[15] Multi-node expert parallelism on Apple Silicon. arXiv:2506.23635.